



数据库系统

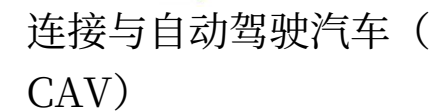
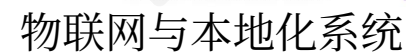
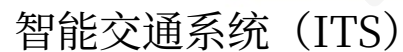
讲师：沈怡文博士

软件与计算机工程系，

韩国 Ajou 大学

chrisshen@ajou.ac.kr

• **研究兴趣:**



- 3/2/2026

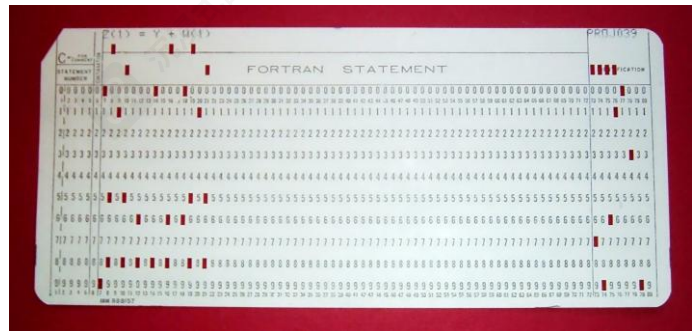
大纲

- 数据库系统历史
- 数据库系统及其应用
- 数据库系统的目的
- 数据视图
- 数据库语言
- 数据库设计
- 数据库引擎
- **数据库架构**
- **数据库用户和管理员**

数据库系统历史



- 1950和20世纪60年代初:
 - 使用磁带进行数据存储
 - 磁带仅提供顺序访问
 - 使用穿孔卡片进行输入
- 20世纪60年代末和70年代:
 - 硬盘可直接访问数据
 - 网络和层次数据模型得到广泛应用
 - Ted Codd定义了关系数据模型
 - 这项工作将赢得ACM图灵奖
 - IBM研究开始System R原型开发
 - 加州大学伯克利分校（迈克尔·石破天惊）开始Ingres原型开发（现PostgreSQL）
 - 甲骨文发布首款商用关系型数据库
 - 高性能（在当时）事务处理



数据库系统历史（续）

- **1980s:**
 - 研究关系原型演变为商业系统
 - SQL成为行业标准
 - 并行和分布式数据库系统
 - 威斯康星 , IBM, Teradata
 - 面向对象数据库系统
- **1990:**
 - 大型决策支持和数据挖掘应用
 - 大型多TB级数据仓库
 - Web商业的兴起

数据库系统历史（续）

- 2000s
 - 大数据存储系统
 - Google BigTable、Yahoo PNuts、Amazon,
 - “NoSQL” 系统。
 - 大数据分析：超越SQL
 - Map reduce和朋友们
- 2010s
 - SQL重载
 - SQL前端到Map Reduce系统
 - 大规模并行数据库系统
 - 多核内存数据库

Database Ecosystem

SQL

Traditional RDBMSs



Generative Value

"Modern" SQL DBs



Not an exhaustive list of companies/segments.
*NoSQL refers to "not-only" SQL - some databases could be in multiple categories.

OLAP Database



Data Warehouse



NoSQL

Document



Graph



Vector



Time-Series*



Search



Key-Value



Multi-Model



Wide Column



中国数据库系统厂商



资料来源：《中国数据库行业研究报告》，艾瑞咨询（2021）、华泰研究

Rank			DBMS	Database Model	Score		
Aug 2025	Jul 2025	Aug 2024			Aug 2025	Jul 2025	Aug 2024
1.	1.	1.	Oracle	Relational, Multi-model ⓘ	1220.70	+3.64	-37.78
2.	2.	2.	MySQL	Relational, Multi-model ⓘ	915.46	-25.26	-111.40
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model ⓘ	754.15	-16.99	-61.02
4.	4.	4.	PostgreSQL	Relational, Multi-model ⓘ	671.25	-9.63	+33.87
5.	5.	5.	MongoDB 📦	Document, Multi-model ⓘ	395.58	-8.25	-25.40
6.	6.	📈7.	Snowflake	Relational	178.90	+2.73	+42.93
7.	7.	📉6.	Redis	Key-value, Multi-model ⓘ	147.19	-2.53	-5.52
8.	8.	📈9.	IBM Db2	Relational, Multi-model ⓘ	127.31	-0.20	+4.30
9.	📈12.	📈15.	Databricks	Multi-model ⓘ	115.82	+7.78	+31.36
10.	📉9.	📉8.	Elasticsearch	Multi-model ⓘ	114.27	-4.56	-15.56
11.	📉10.	📉10.	SQLite	Relational	112.59	-2.84	+7.80
12.	📉11.	📉11.	Apache Cassandra	Wide column, Multi-model ⓘ	108.51	-0.24	+11.51
13.	13.	📈14.	MariaDB 📦	Relational, Multi-model ⓘ	93.59	-1.85	+7.06
14.	14.	📉12.	Microsoft Access	Relational	87.76	-2.71	-8.61
15.	15.	📈17.	Amazon DynamoDB	Multi-model ⓘ	83.48	-0.67	+14.57
16.	16.	16.	Microsoft Azure SQL Database	Relational, Multi-model ⓘ	75.84	-0.67	+0.81
17.	17.	📈19.	Apache Hive	Relational	71.03	-4.71	+15.79
18.	18.	📉13.	Splunk	Search engine	69.77	+0.25	-26.33
19.	19.	📉18.	Google BigQuery	Relational	65.18	+0.99	+9.65
20.	20.	📈21.	Neo4j	Graph	54.47	-0.16	+10.58

来源: <https://db-engines.com/en/ranking>

DB-Engines排名

趋势:

h
t
t

ps://db-engines.com/en/ranking_trend

相关顶级会议

- ACM SIGMOD (数据管理特别兴趣小组) : <https://sigmod.org/>

- ACM SIGIR (信息检索特别兴趣小组) : <https://sigir.org/>

- VLDB (超大型数据库) : <https://www.vldb.org/>

- 2025 [2025](#) [2025](#) :
- 2026 [2026](#)

<https://vldb.org/?program-schedule-> : <https://vldb.org/> /

- **ACM CIKM (Conference on Information and Knowledge Management):**

- <http://www.cikmconference.org/>

- 2024: <https://cikm2024.org/>

- 2025: <https://cikm2025.org/>

数据库系统

- DBMS（数据库管理系统）包含有关特定企业的信息
 - 相互关联的数据集合
 - 用于访问数据的一组程序
 - 一个既方便又高效的使用环境
- 数据库系统用于管理以下类型的数据集合：
 - 极具价值
 - 相对较大
 - 被多个用户和应用程序同时访问。
- 现代数据库系统是一个复杂的软件系统，其任务是管理一个庞大而复杂的数据集合。
- 数据库影响着我们生活的方方面面

A Day In Susan's Life

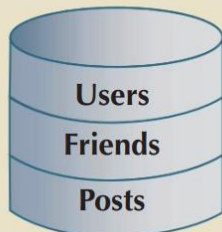
See how many databases she interacts with each day

Before leaving for work,
Susan checks her
Facebook and
Twitter accounts



Where is the data about the
friends and groups stored?

Where are the "likes" stored
and what would they be
used for?



On her lunch break,
she picks up her
prescription at the
pharmacy



Where is the pharmacy
inventory data stored?

What data about each
product will be in the
inventory data?

What data is kept about
each customer and where
is it stored?



After work, Susan
goes to the grocery
store



Where is the product
data stored?

Is the product quantity in
stock updated at checkout?

Does she pay with a credit
card?



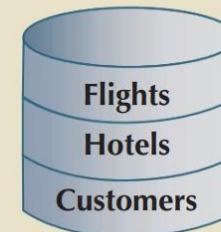
At night, she plans for a trip
and buys airline tickets and
hotel reservations online



Where does the online
travel website get the
airline and hotel data from?

What customer data would
be kept by the website?

Where would the customer
data be stored?



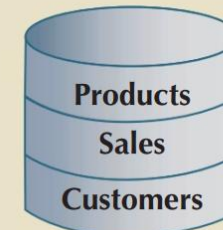
Then she makes a few
online purchases



Where are the product
and stock data stored?

Where does the system get
the data to generate product
"recommendations" to the
customer?

Where would credit card
information be stored?



数据库应用实例

- 企业信息
 - 销售：客户、产品、采购
 - 会计：付款、收据、资产
 - 人力资源：员工信息、薪资、工资税。
- 制造业：生产管理、库存、订单、供应链。
- 银行与金融
 - 客户信息、账户、贷款和银行交易。
 - 信用卡交易
 - 金融：金融工具（例如股票和债券）的销售和购买；存储实时市场数据
- 大学：注册、成绩

数据库应用实例（续）

- 航空公司：预订、时刻表
- 电信：通话、短信及数据使用记录，生成月账单，维护预付费通话卡的余额
- 基于网络的服务
 - 在线零售商：订单跟踪、个性化推荐
 - 在线广告
- 文档数据库
- 导航系统：用于维护兴趣地点的位置，以及道路、铁路系统、公交车等的精确路线。

数据库系统的目的

- 在早期，数据库应用程序是直接建立在文件系统之上的，这导致了：
 - 数据冗余和不一致性：数据以多种文件格式存储，导致不同文件中信息的重复
 - 访问数据困难
 - 需要为每个新任务编写新程序
 - 数据隔离：多个文件和格式
 - 完整性问题
 - 完整性约束（例如，账户余额 > 0 ）会“隐藏”在程序代码中，而不是明确声明
 - 难以添加新的约束或修改现有约束

数据库系统的目的（续）

- 更新的原子性
 - 故障可能使数据库处于部分更新已完成但状态不一致的状态
 - 示例：从一个账户向另一个账户转账要么完全完成，要么根本不发生
- 多个用户并发访问
 - 并发访问对于性能是必要的
 - 不受控的并发访问可能导致不一致
 - 例如：两个人同时读取一个余额（比如100），并进行取款（每人50）的更新操作
- 安全问题
 - 难以提供对某些数据访问，但不对所有数据访问
- 数据库系统为上述所有问题提供了解决方案

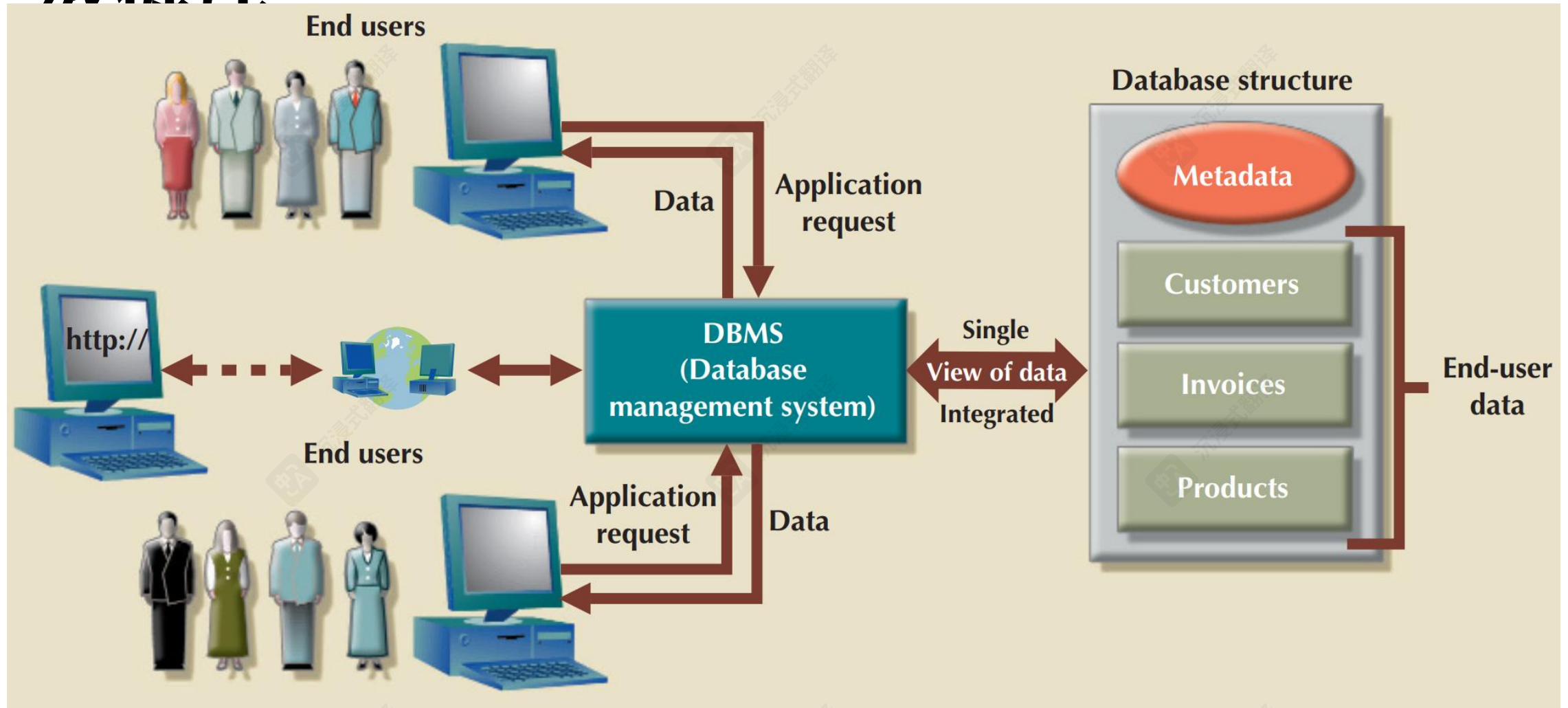
大学数据库示例

- 在本文本中，我们将使用一个大学数据库来阐述所有概念
- 数据包括以下信息：
 - 学生
 - 教师
 - 课程
- 应用程序示例：
 - 添加新学生、教师和课程
 - 为学生注册课程，并生成班级花名册
 - 为学生评分，计算平均绩点（GPA）并生成成绩单

数据视图

- 数据库系统是一组相互关联的数据以及一组允许用户访问和修改这些数据的程序。
- 数据库系统的主要目的是为用户提供数据的抽象视图。
 - **数据模型**
 - 描述数据、数据关系、数据语义和一致性约束的概念工具集合。
 - **数据抽象**
 - 将数据结构的复杂性对用户隐藏，通过多级数据抽象来表示数据库中的数据。

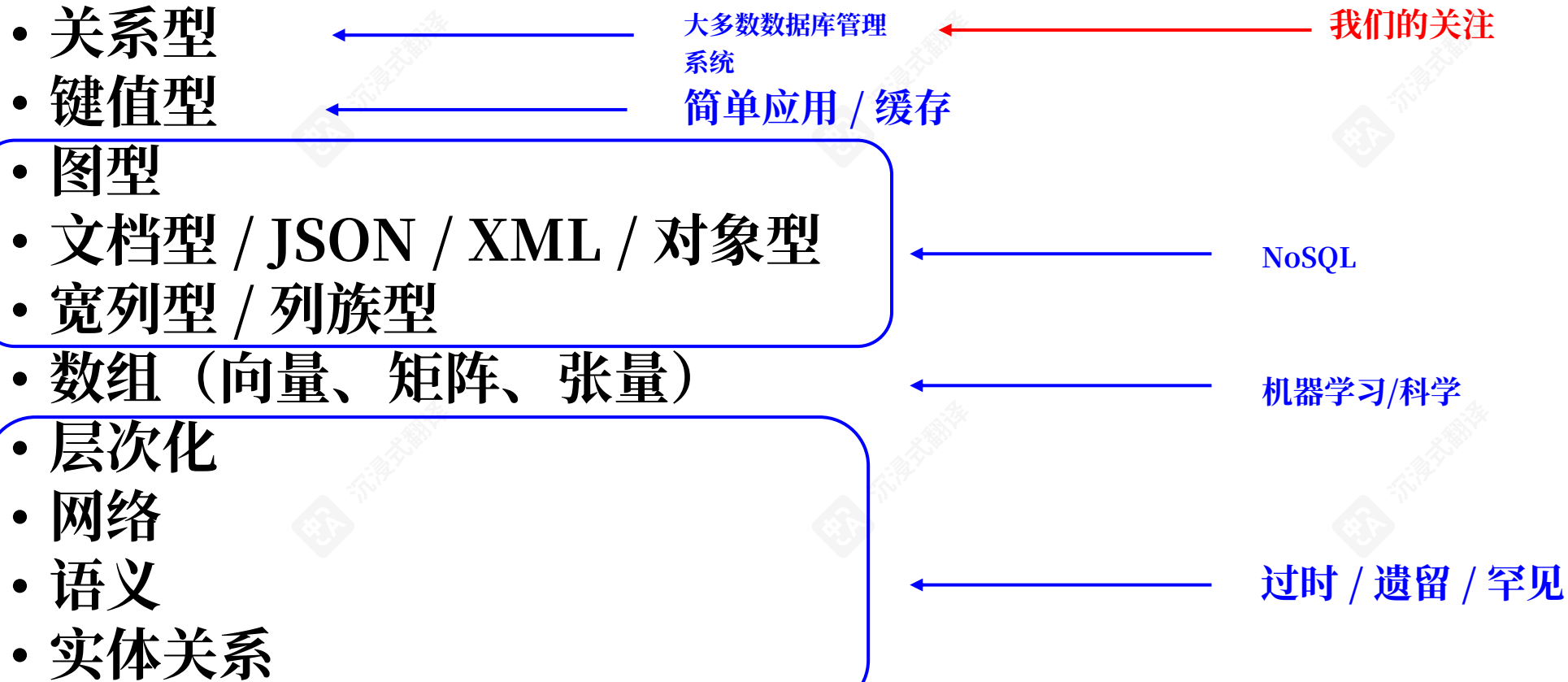
终端用户与 数据库



数据模型

- 关系模型
- 实体-关系数据模型（主要用于数据库设计）
- 基于对象的数据模型（面向对象和面向对象-关系）
- 半结构化数据模型（XML）
- 其他较旧模型：
 - 网络模型
 - 层次模型

数据模型列表



早期的数据库管理系统

- Ted Codd是IBM研究所在20世纪60年代末的数学家。
- Codd看到IBM的开发人员每次数据库模式或布局发生变化时都要重写数据库程序。
- 于1969年提出了关系模型。
 - <https://dl.acm.org/doi/10.1145/1558334.1558336>
 - <https://dl.acm.org/doi/10.1145/362384.362685>

DERIVABILITY, REDUNDANCY AND CONSISTENCY OF RELATIONS STORED IN LARGE DATA BANKS

E. F. Codd
Research Division
San Jose, California

ABSTRACT: The large, integrated data banks of the future will contain many relations of various degrees in stored form. It will not be unusual for this set of stored relations to be redundant. Two types of redundancy are defined and discussed. One type may be employed to improve accessibility of certain kinds of information which happen to be in great demand. When either type of redundancy exists, those responsible for control of the data bank should know about it and have some means of detecting any "logical" inconsistencies in the total set of stored relations. Consistency checking might be helpful in tracking down unauthorized (and possibly fraudulent) changes in the data bank contents.

RJ 599 (# 12343)
August 19, 1969

A Relational Model of Data for Large Shared Data Banks

E. F. Codd
IBM Research Laboratory, San Jose, California

Future users of large data banks must be protected from having to know how the data is organized in the machine (the internal representation). A prompting service which supplies such information is not a satisfactory solution. Activities of users at terminals and most application programs should remain unaffected when the internal representation of data is changed and even when some aspects of the external representation are changed. Changes in data representation will often be needed as a result of changes in query, update, and report traffic and natural growth in the types of stored information.

Existing noninferential, formatted data systems provide users with tree-structured files or slightly more general network models of the data. In Section 1, inadequacies of these models are discussed. A model based on n -ary relations, a normal form for data base relations, and the concept of a universal data sublanguage are introduced. In Section 2, certain operations on relations (other than logical inference) are discussed and applied to the problems of redundancy and consistency in the user's model.

KEY WORDS AND PHRASES: data bank, data base, data structure, data organization, hierarchies of data, networks of data, relations, derivability, redundancy, consistency, composition, join, retrieval language, predicate calculus, security, data integrity
CR CATEGORIES: 3.70, 3.73, 3.75, 4.20, 4.22, 4.29

1. Relational Model and Normal Form

1.1. INTRODUCTION

This paper is concerned with the application of elementary relation theory to systems which provide shared access to large banks of formatted data. Except for a paper by Childs [1], the principal application of relations to data systems has been to deductive question-answering systems. Levein and Maron [2] provide numerous references to work in this area.

In contrast, the problems treated here are those of *data independence*—the independence of application programs and terminal activities from growth in data types and changes in data representation—and certain kinds of *data inconsistency* which are expected to become troublesome even in nondeductive systems.

The relational view (or model) of data described in Section 1 appears to be superior in several respects to the graph or network model [3, 4] presently in vogue for non-inferential systems. It provides a means of describing data with its natural structure only—that is, without superimposing any additional structure for machine representation purposes. Accordingly, it provides a basis for a high level data language which will yield maximal independence between programs on the one hand and machine representation and organization of data on the other.

A further advantage of the relational view is that it forms a sound basis for treating derivability, redundancy, and consistency of relations—these are discussed in Section 2. The network model, on the other hand, has spawned a number of confusions, not the least of which is mistaking the derivation of connections for the derivation of relations (see remarks in Section 2 on the "connection trap").

Finally, the relational view permits a clearer evaluation of the scope and logical limitations of present formatted data systems, and also the relative merits (from a logical standpoint) of competing representations of data within a single system. Examples of this clearer perspective are cited in various parts of this paper. Implementations of systems to support the relational model are not discussed.

1.2. DATA DEPENDENCIES IN PRESENT SYSTEMS

The provision of data description tables in recently developed information systems represents a major advance toward the goal of data independence [5, 6, 7]. Such tables facilitate changing certain characteristics of the data representation stored in a data bank. However, the variety of data representation characteristics which can be changed *without logically impairing some application programs* is still quite limited. Further, the model of data with which users interact is still cluttered with representational properties, particularly in regard to the representation of collections of data (as opposed to individual items). Three of the principal kinds of data dependencies which still need to be removed are: ordering dependence, indexing dependence, and access path dependence. In some systems these dependencies are not clearly separable from one another.

1.2.1. Ordering Dependence. Elements of data in a data bank may be stored in a variety of ways, some involving no concern for ordering, some permitting each element to participate in one ordering only, others permitting each element to participate in several orderings. Let us consider those existing systems which either require or permit data elements to be stored in at least one total ordering which is closely associated with the hardware-determined ordering of addresses. For example, the records of a file concerning parts might be stored in ascending order by part serial number. Such systems normally permit application programs to assume that the order of presentation of records from such a file is identical to (or is a subordering of) the



关系模型

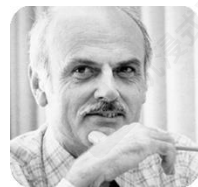
关系模型

- 所有数据都存储在不同的表格中。
- 关系模型中的表格数据示例

列

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

Rows



Ted Codd

图灵奖1981

<https://zh.wikipedia.org/wiki/埃德加·科德>

一个示例关系型数据库

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

抽象层次

- 物理层：描述记录（例如，讲师）的存储方式。
- 逻辑层：描述数据库中存储的数据以及数据之间的关系。

```
type instructor = record
```

```
    ID : string;
```

```
    department : string; 工资 : 整
```

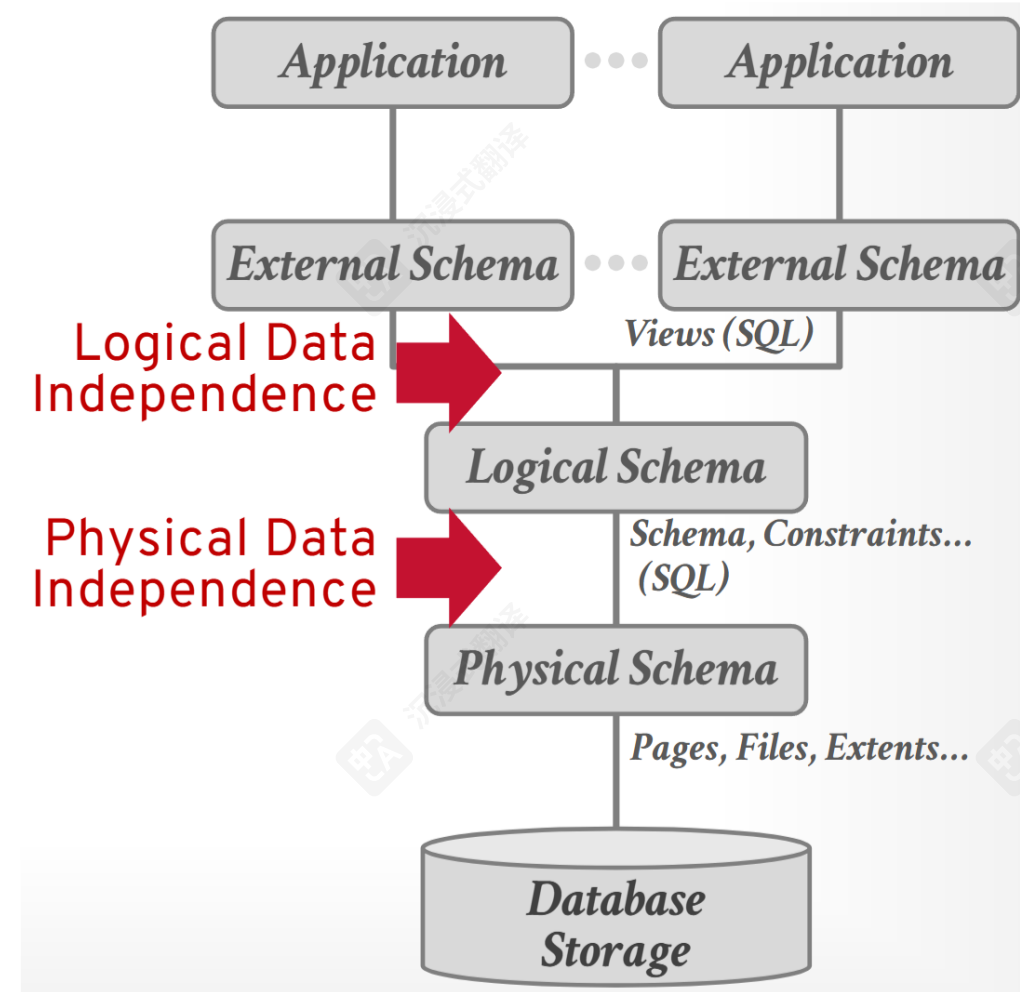
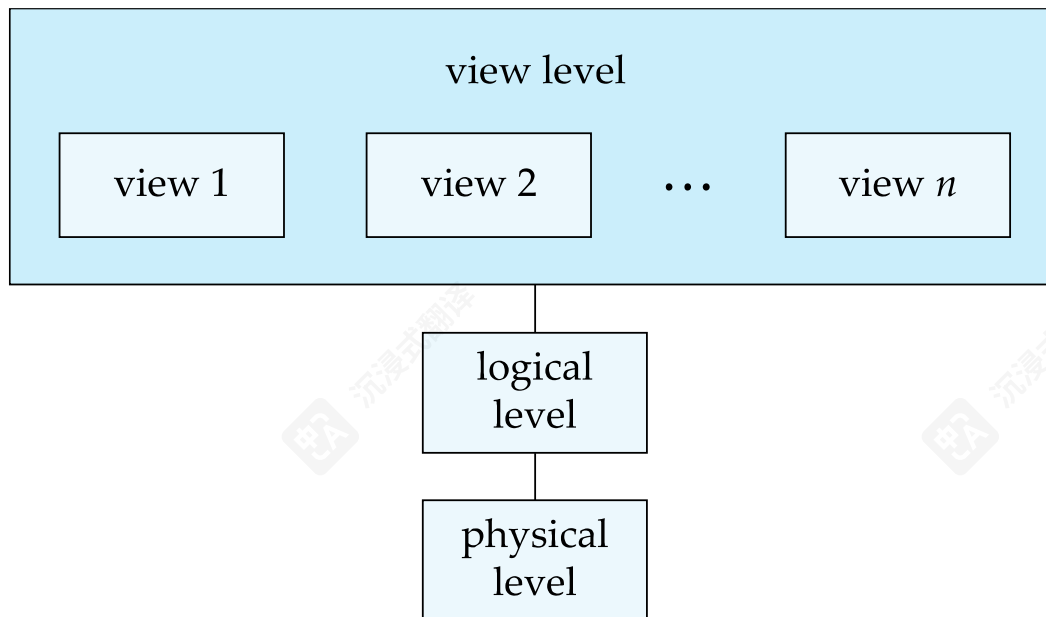
```
数;
```

```
end;
```

- 视图级别：应用程序隐藏数据类型的细节。视图也可以为了安全目的隐藏信息（例如员工的工资）。

数据视图

- 一个数据库系统的架构



实例和模式

- 类似于编程语言中的类型和变量
- 逻辑模式——数据库的整体逻辑结构
 - 示例：数据库包含银行一组客户和账户的信息以及它们之间的关系
 - 类似于程序中变量的类型信息
- 物理模式——数据库的整体物理结构

实例 • 实例 – 数据库在特定时间点的实际内容

- 类似于变量的值

物理数据独立性

物理数据独立性

- 物理数据独立性——即修改物理模式而不改变逻辑模式的能力
 - 应用程序依赖于逻辑模式
 - 通常，不同层级和组件之间的接口应该定义良好，以便某些部分的变化不会严重影响其他部分。

数据定义语言 (DDL)

数据定义语言

- 定义数据库架构的规范符号

示例:

```
create table instructor (
```

ID

name

部门_名称

薪资

char(5),

varchar(20),

varchar(20),

numeric(8,2))

编译器

数据字典

- DDL编译器生成一组表模板，并存储在数据字典中
- 数据字典包含元数据（即关于数据的数据）
 - 数据库架构
 - 完整性约束
 - 完整性约束
 - 主键 (ID唯一标识讲师)
 - 授权
 - 授权
 - 谁可以访问什么

数据操作语言 (DML)

数据操作语言

- 一种用于访问和更新由适当数据模型组织的数据的语言
 - DML 也称为查询语言
- 数据操作语言基本上有两种类型
 - 过程化 DML —— 要求用户指定需要哪些数据以及如何获取这些数据
 - 声明式 DML —— 要求用户指定需要哪些数据，而无需指定如何获取这些数据
- 声明式DML通常比过程式DML更容易学习和使用。
- 声明式DML也被称为非过程式DML。
- DML中涉及信息检索的部分称为查询语言。

SQL查询语言

- SQL（结构化查询语言）是一种非过程化语言。一条查询以一个或多个表（可能只有一个）作为输入，并始终返回一个表。
- 查找计算机科学系所有讲师的示例

```
select name from instructor where dept_name = '计算机科学'
```
- SQL不是图灵机等价语言
- 为了能够计算复杂函数，SQL通常嵌入在某些高级语言中
- 应用程序通常通过以下方式访问数据库：
 - 语言扩展以支持嵌入式SQL
 - 应用程序接口（例如ODBC/JDBC），允许将SQL查询发送到数据库

开放数据库连接（ODBC）

应用程序程序访问数据库

- 非过程化查询语言如SQL并不像通用图灵机那样强大。
 - SQL不支持用户输入、显示输出或网络通信等操作。
- 此类计算和操作必须在宿主语言（如C/C++、Java或Python）中编写，并嵌入SQL查询以访问数据库中的数据。
- 应用程序——是指以这种方式与数据库交互的程序。

宿主语言中的 SQL 示例

Python

```
import mysql.connector

mydb =
mysql.connector.connect( host="
localhost", user="yourusername",
password="yourpassword" )

mycursor = mydb.cursor()

mycursor.execute("CREATE
DATABASE mydatabase")
```

• C++

```
MYSQL *mysql_init(MYSQL *);
```

```
MYSQL mysql_real_
connect( MYSQL connection,
const char *host, const char
*username, const char
*password, const char
*database_name, unsigned int
port, const char *unix_socket_
name, unsigned int flags );
```

```
int mysql_query(MYSQL *connection, const
char *query);
```

数据库设计

- 设计数据库整体结构的过程：
- 逻辑设计——确定数据库模式。数据库设计要求我们找到一个“好”的关系模式集合。
 - 业务决策——我们应该在数据库中记录哪些属性？
 - 计算机科学决策——我们应该有哪些关系模式，以及属性应该如何分布在各种关系模式中？
- 物理设计——确定数据库的物理布局

数据库引擎

- 数据库系统被划分为多个模块，每个模块负责整体系统的某项职责。
- 数据库系统的功能组件可以分为
 - 存储管理器，
 - 查询处理器组件，
 - 事务管理组件。

存储管理器

存储管理器

- 一个提供数据库中存储的低级数据与应用程序程序和系统提交的查询之间接口的程序模块。
- 存储管理器负责以下任务：
 - 与操作系统文件管理器交互
 - 高效地存储、检索和更新数据
- 存储管理器组件包括：
 - 授权与完整性管理器
 - 事务管理器
 - 文件管理器
 - 缓冲区管理器

存储管理器（续）

- 存储管理器作为物理系统实现的一部分，实现了多种数据结构：
 - 数据文件——存储数据库本身
 - 数据字典——存储关于数据库结构的元数据，特别是数据库模式。
 - 索引——可以提供对数据项的快速访问。数据库索引为包含特定值的数据项提供指针。

查询处理器

查询处理器

- 查询处理器组件包括：

DDL解释器 • DDL解释器 -- 解释DDL语句，并将定义记录在数据字典中。

DML 编译器 • DML编译器 -- 将查询语言中的DML语句翻译成查询执行引擎理解的低级指令组成的执行计划。

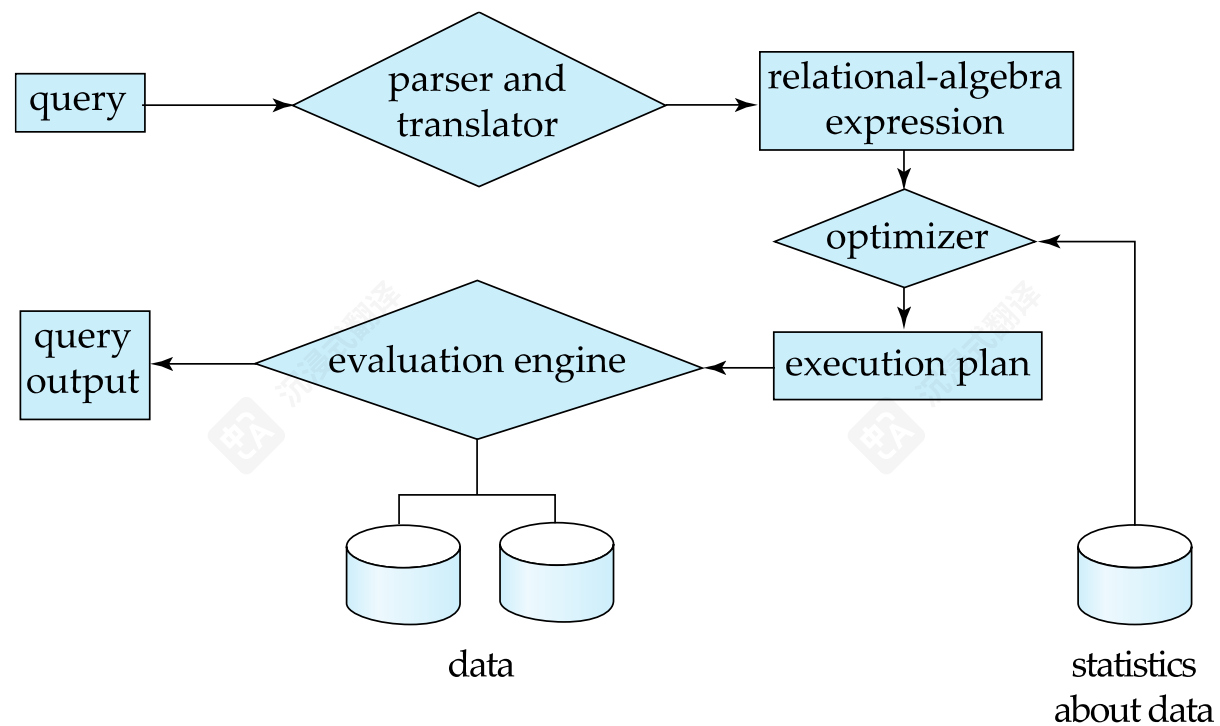
- DML编译器执行查询优化；也就是说，它从各种备选方案中选择成本最低的执行计划。

- 查询执行引擎 -- 执行生成的低级指令由 DML 编译器处理。

查询评估引擎

查询处理

- 1. 解析与翻译
- 2. 优化
- 3. 评估



事务 事务管理

- 事务是一组执行数据库应用程序中单一逻辑功能的操作
- 事务管理组件确保数据库即使在系统故障（例如，电源故障和操作系统崩溃）和事务故障的情况下也能保持一致（正确）状态
- 并发控制管理器控制并发事务之间的交互，以确保数据库的一致性

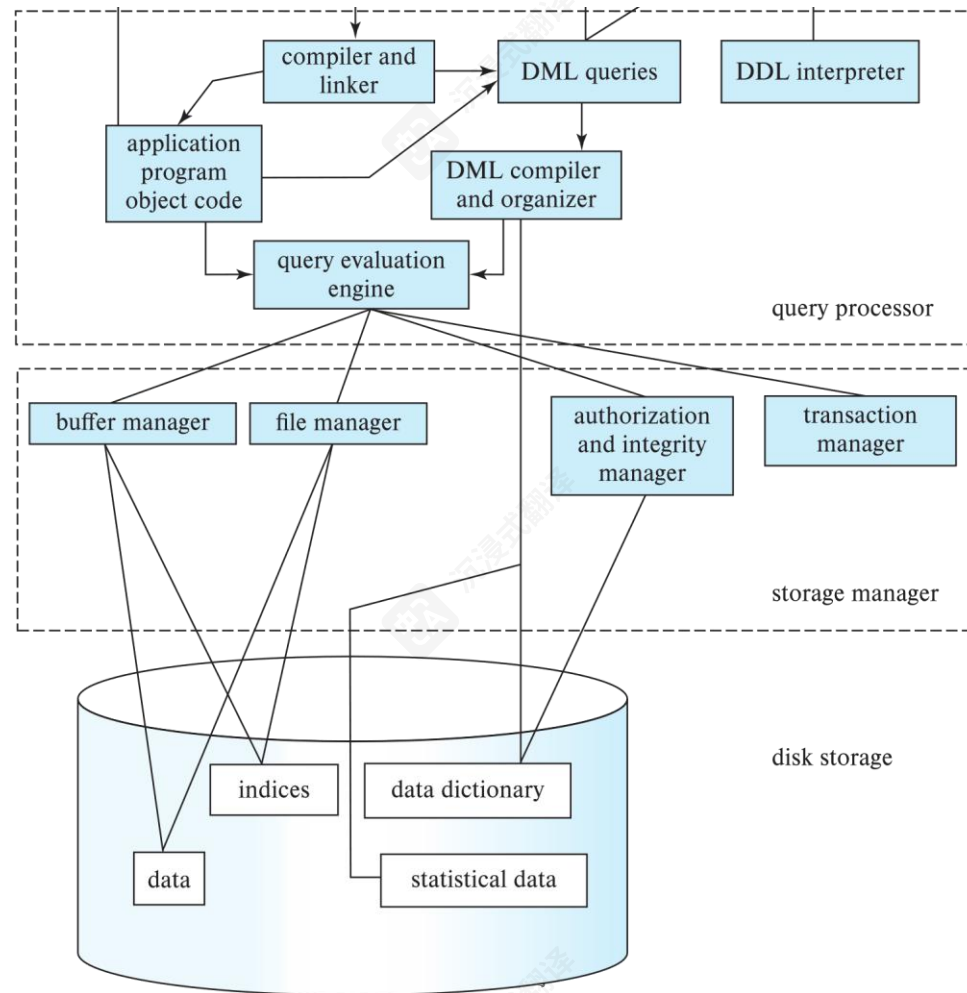
并发控制管理器

数据库架构

数据库架构

- 集中式数据库
 - 一到几个核心，共享内存
- 客户端-服务器，
 - 一台服务器机器代表多台客户端机器执行工作。
- 并行数据库
 - 许多核心共享内存
 - 共享磁盘
 - 无共享
- 分布式数据库
 - 地域分布
 - 模式/数据异构性

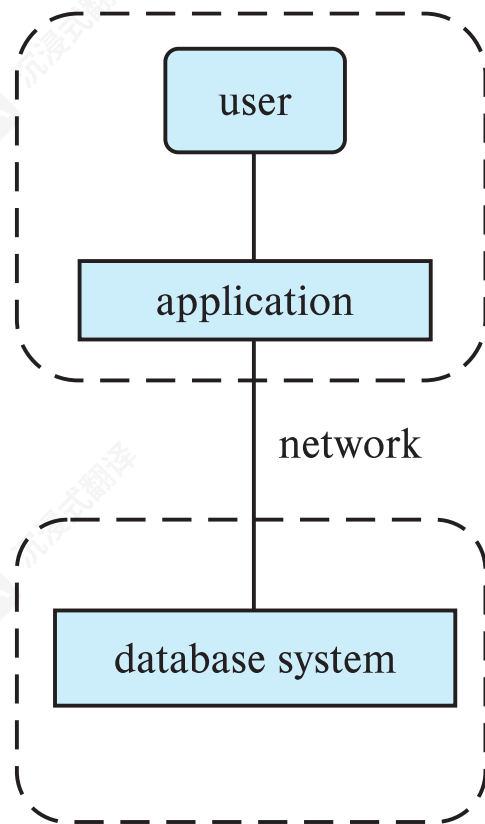
数据库架构 (集中式/共享内存)



数据库应用

- 数据库应用通常分为两部分或三部分
- 两层架构——应用程序驻留在客户端机器上，在那里它调用服务器机器上的数据库系统功能
- 三层架构——客户端机器充当前端，不包含任何直接的数据库调用
 - 客户端端与应用服务器通信，通常通过表单界面进行
 - 应用服务器反过来与数据库系统通信以访问数据

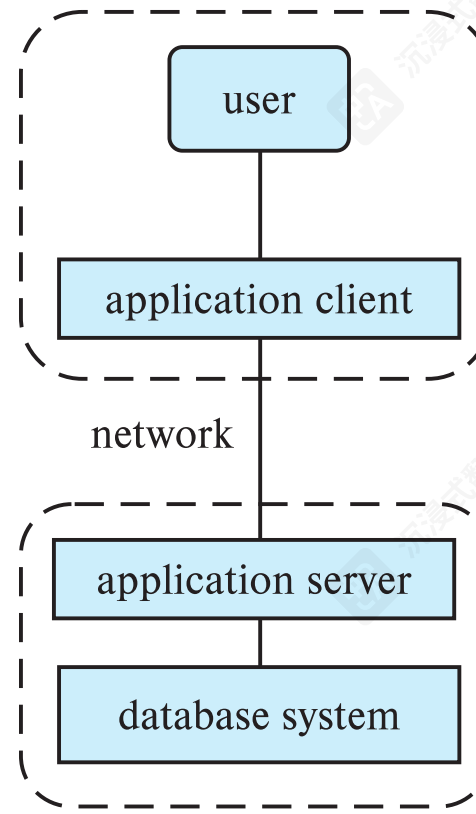
两层和三层架构



(a) Two-tier architecture

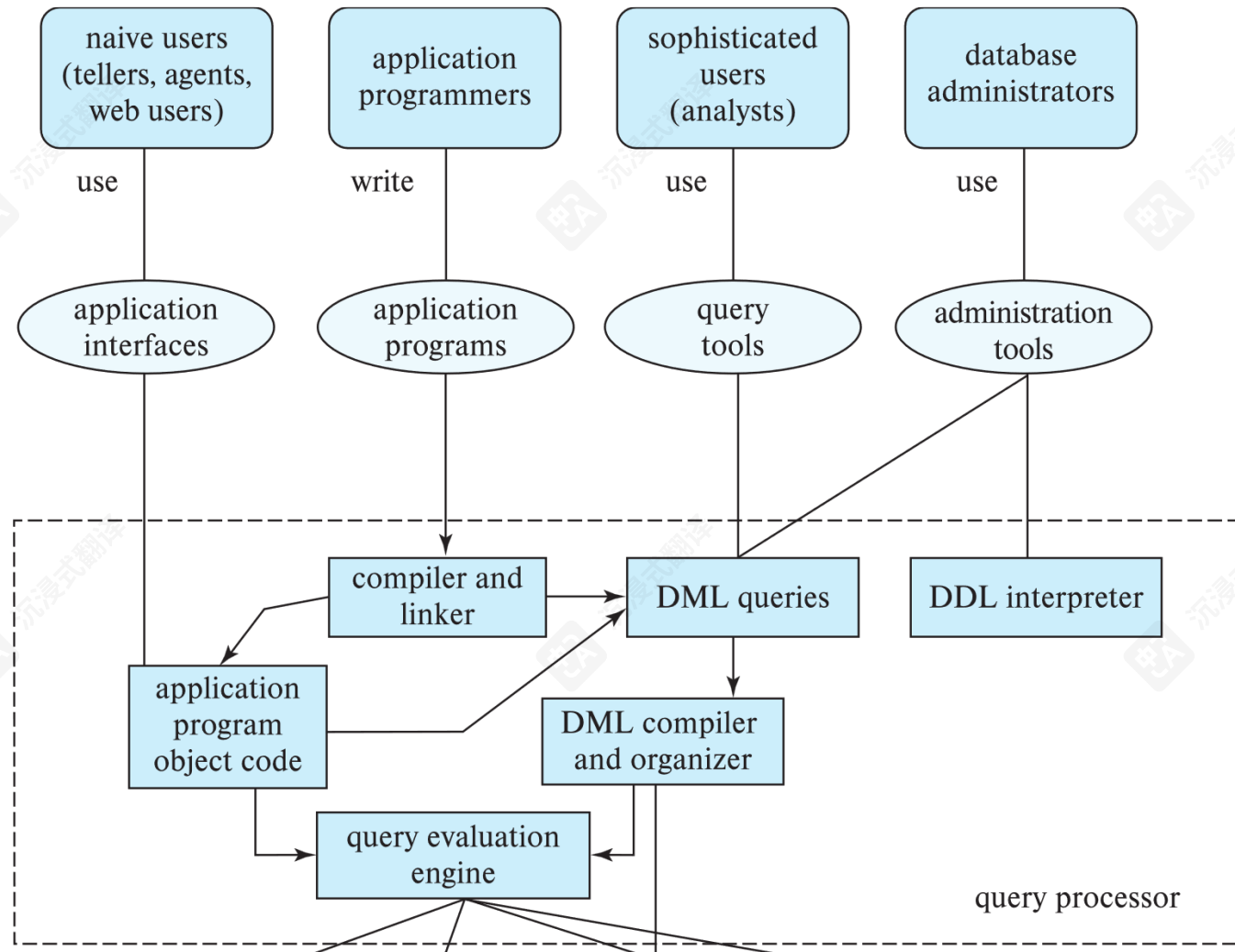
client

server



(b) Three-tier architecture

数据库用户



数据库管理员

- 拥有系统中央控制权的人称为数据库管理员（DBA）。数据库管理员（DBA）的职责包括：
 - 模式定义
 - 存储结构和访问方法定义
 - 模式和物理组织的修改
 - 授权数据访问
 - 常规维护
 - 定期备份数据库
 - 确保有足够的可用磁盘空间以进行正常操作，并在需要时升级磁盘空间
- 监控数据库上运行的作业

课程信息与后勤

课程概要

- 本课程旨在提供对数据库概念、设计、实施和管理全面的理解。
- 数据库是现代计算的核心，对于在从Web应用到大型企业系统等各种应用中存储、检索和管理数据至关重要。
- 我们将学习多个主题，例如数据模型和实体-关系模型、关系数据库设计、SQL编程、索引和查询优化、数据库安全和管理、NoSQL数据库、大数据和分布式数据库等。

涵盖主题（暂定）

- 关系型语言、结构化查询语言（SQL）
- 数据库设计：实体-关系模型
- 关系型数据库设计
- 应用设计与开发：复杂数据类型
- 数据分析
- 数据存储结构
- 索引和哈希
- 查询处理；事务

课程信息。

- 日期和时间：周二， 10:00-12:45
- 地点：연암관 902
- 办公时间（按需）：
 - 时间：周一 17:00 - 17:30, 周二 17:00 - 17:30
 - 办公室：팔달관 (Paldal Hall) 1011
 - 也可通过视频会议参与。

课程学习平台



- 平台： Ajou Blackboard (Bb)
 - 网址： <https://eclass2.ajou.ac.kr/ultra/institution-page>
 - 提供安卓和 iOS 应用（<https://linktr.ee/ajoubb>）。
 - 讲义、资料、测验、作业等将定期更新。
 - 提供录播课程。

• 问答

- 鼓励使用Ajou Bb提问。
- 讲师和助教会尝试回答你的问题。
- 你的问题也能帮助他人提升理解。



QnA



Visible to students ▼

Please leave a question for this course.

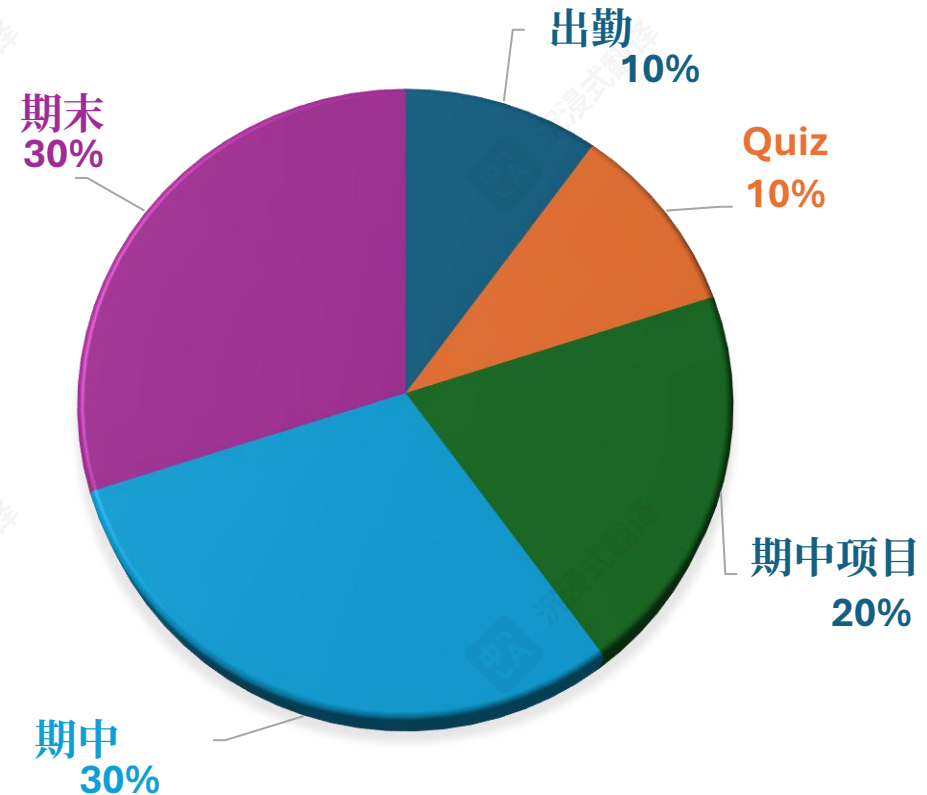
通讯

- 我们使用Ajou Bb和电子邮件进行通讯：
 - 请说明您问题的目的。
 - ☐好的标题：如何使用不同的参数在Python中读取csv文件？
 - ☐不好的标题：关于讲座的问题
 - 如果使用电子邮件，请在邮件中明确注明您的姓名和学号。
- 使用英语/中文进行交流，在 Ajou 提问，以及所有作业。
Bb，以及所有作业。
 - 如果在作业和考试中使用其他语言，你可能会得到零分。

评分政策

- 助教：
 - 邮箱：
- 评价方式：
 - 测验；
 - 期中项目；
 - 期中和期末考试。
- 评分政策：

评分政策



考勤

- 使用电子考勤系统。

- 指南: <https://www.ajou.ac.kr/kr/ajou/notice.do?mode=view&articleNo=192571&article.offset=0&文章限制=10&sr搜索值=%EA%B3%84%EC%A0%88>

- 4 缺勤 -> 不及格

- 2 迟到/早退 -> 1 缺勤

- 因合理原因缺勤并提供证明:

- 生病、军事训练、家庭原因等。详细说明:

<https://www.ajou.ac.kr/kr/ajou/notice.do?mode=view&articleNo=324578&article.offset=0&articleLimit=10>

- 请在EAS系统中缺勤后7天内提交适当证明。

EAS考勤



아주대학교 통합 모바일

전자출석부 사용법

학생증 태그 방식 사용 불가

블루투스 비콘 인증방법

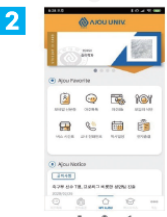
1



앱 다운로드

스토어 검색 창에 '아주대학교 통합 모바일'을 입력하여 모바일 앱을 다운로드

2



앱 실행

블루투스 및 위치서비스를 반드시 ON으로 설정!

인증번호 출석방법

3



출석 인증 전

강의 시작 15분 전부터 출결 화면이 실행됩니다. 강의실 내부에서 '출석하기' 버튼을 눌러주세요.

4



출석 인증 후

사용자의 GPS를 분석하여 강의실 내부일 경우에만 출석인증이 가능합니다.

인증번호 출석

인증번호 출석을 진행하는 경우, '인증번호' 버튼이 활성화됩니다. 인증번호 입력창으로 이동!

마무리

교수님이 지정한 인증번호를 입력한 경우, 출석으로 처리됩니다.



자세한 매뉴얼은 학교 공지사항 참조
(포털-공지사항-전자출결로 검색)

- **出席：**上课开始前15分钟和结束后10分钟内。
 - 基于BLE的考勤检查由点击智能手机App完成。
 - 基于验证码的考勤检查。
- **迟到：**上课开始后10分钟至30分钟内。
- 请确保检查EAS中每堂课的出勤状态。
- 任何未定义状态 (미인증) 将被视为缺席。

测验与作业

- 测验

- 总共我们将有4次测验，每次10道题。
- 测验从第3周开始（每两周一次）。
- 格式：
 - 单选题（每题只有一个正确答案）。

- 学期项目

- 我们将有1个学期项目。
- 提交内容：
 - 项目报告
 - 源代码（包括运行说明文档和代码结构）
 - 演示幻灯片（powerpoint文件）
 - 如果可能，一个演示（视频片段）

期中与期末考试

• 期中

- 形式：课堂闭卷笔试
- 时间：4月14日，上午10:00 - 11:00
- 时长：60分钟
- 允许携带考试笔记：1面A4手写
 - 必须是自己手写
 - 不是从别人那里抄的

• 期末考试

- 形式：课堂闭卷笔试
- 时间：6月9日，上午10:00 - 11:00
- 时长：60分钟
- 允许携带考试笔记：单面A4手写
 - 必须是自己手写
 - 不能是复制品

日历

2026年3月							2026年4月							2026年5月						
S	M	T	W	T	F	S	S	M	T	W	T	F	S	S	M	T	W	T	F	S
1	2	3	4	5	6	7				1	2	3	4						1	2
8	9	10	11	12	13	14	5	6	7	8	9	10	11	3	4	5	6	7	8	9
15	16	17	18	19	20	21	12	13	14	15	16	17	18	10	11	12	13	14	15	16
22	23	24	25	26	27	28	19	20	21	22	23	24	25	17	18	19	20	21	22	23
29	30	31					26	27	28	29	30			24	25	26	27	28	29	30
														31						
2025年6月																				
S	M	T	W	T	F	S														
	1	2	3	4	5	6														
7	8	9	10	11	12	13														
14	15	16	17	18	19	20														
21	22	23	24	25	26	27														
28	29	30																		

团队项目

- 使用移动模拟器设计数据库。
- “城市交通模拟” (SUMO): <https://eclipse.dev/sumo/>
 - 微观和连续交通模拟
 - 文档: <https://sumo.dlr.de/docs/index.html>
- 使用 Python 接口控制模拟, 称为 TraCI (交通控制接口) 。
- 使用 Python 接口结合 SQL 代码来设计数据库。
- 每个团队由 2 名学生组成。

课程项目主题（待详细说明）

- 出租车（或拼车）信息数据库系统

出租车（或共乘车辆）信息数据库系统

- 拼车用户信息数据库系统

共乘用户信息数据库系统

- 拼车平台管理系统

共享乘车平台管理系统

- 车辆导航信息数据库系统

车辆导航信息数据库系统

- 城市道路交通数据库系统

城市道路交通数据库系统

基于LLM的工具使用政策

- 基于LLM的工具（例如ChatGPT和Gemini）禁止在任何测验、作业和考试中直接复制生成的内容。
- 对于期中项目报告，如果任何文本是由LLM工具生成或修改的，应在报告末尾注明修改位置、所使用的LLM工具类型，并确保修改后的文本符合原始写作意图。
 - 需要提供的信息：工具名称、版本、提示输入内容

学术不端行为

- **作弊**
 - 作弊包括在学术作业中存在欺诈、欺骗或不诚实行为，或使用或试图使用在所涉及的学术作业背景下被禁止或不适当材料，或协助他人使用这些材料。
- **剽窃**
 - 剽窃包括未经承认其来源而使用他人产生的智力成果。
- **提供虚假信息、捏造或篡改信息**
 - 提供虚假信息、未能诚实表明身份、伪造或篡改信息并声称其合法，或向教师或任何其他大学官员在学术环境中提供虚假或误导性信息。
- **知识产权盗窃或损坏**
 - 破坏或盗窃他人的作品、不当访问或通过电子方式干扰他人的财产或大学的财产，或在不经批准的情况下获取考试或作业的副本。
- **大学文件篡改**
 - 伪造教师签名、向另一所机构或雇主提交篡改的成绩单、将个人姓名放在他人的作品上，或虚假篡改已评分的考试或作业。

学术不端行为

- **作弊**
 - 作弊是指在学术作业中存在欺诈、欺骗或不诚实的行为，或使用或试图使用相关材料，或协助他人使用与学术作业内容不符或不适当的材料。
- **剽窃**
 - 剽窃是指未经许可或未注明出处而使用他人创作的智力成果。
- **虚假信息、不实陈述及伪造或篡改信息**
 - 提供虚假信息，未能诚实地表明身份，伪造或篡改信息并将其视为合法信息，或在学术环境中向教师或任何其他大学官员提供虚假或误导性信息。
- **盗窃或破坏知识产权**
 - 破坏或窃取他人作品，不当获取或以电子方式干扰他人或大学的财产，或在考试或作业批准发布之前获取其副本。
- **篡改大学文件**
 - 伪造教师签名、向其他机构或雇主提交篡改的成绩单、在他人作业上署名、或伪造已评分的考试或作业。

数据库系统 感谢!

问答?

讲师：沈怡文，博士
软件与计算机工程系，
韩国 Ajou 大学
chrisshen@ajou.ac.kr